МИНОБРНАУКИ РОССИИ
САНКТ-ПЕТЕРБУРГСКИЙ ГОСУДАРСТВЕННЫЙ
ЭЛЕКТРОТЕХНИЧЕСКИЙ УНИВЕРСИТЕТ
«ЛЭТИ» ИМ. В.И. УЛЬЯНОВА (ЛЕНИНА)
Кафедра МО ЭВМ

ЛАБОРАТОРНАЯ РАБОТА №1 ВАРИАНТ 1Р

по дисциплине «Построение и анализ алгоритмов»

Тема: Поиск с возвратом

Студентка гр. 4345 _____ Левкович Ю.С.

Преподаватель _____ Жангиров Т.Р.

Санкт-Петербург
2025

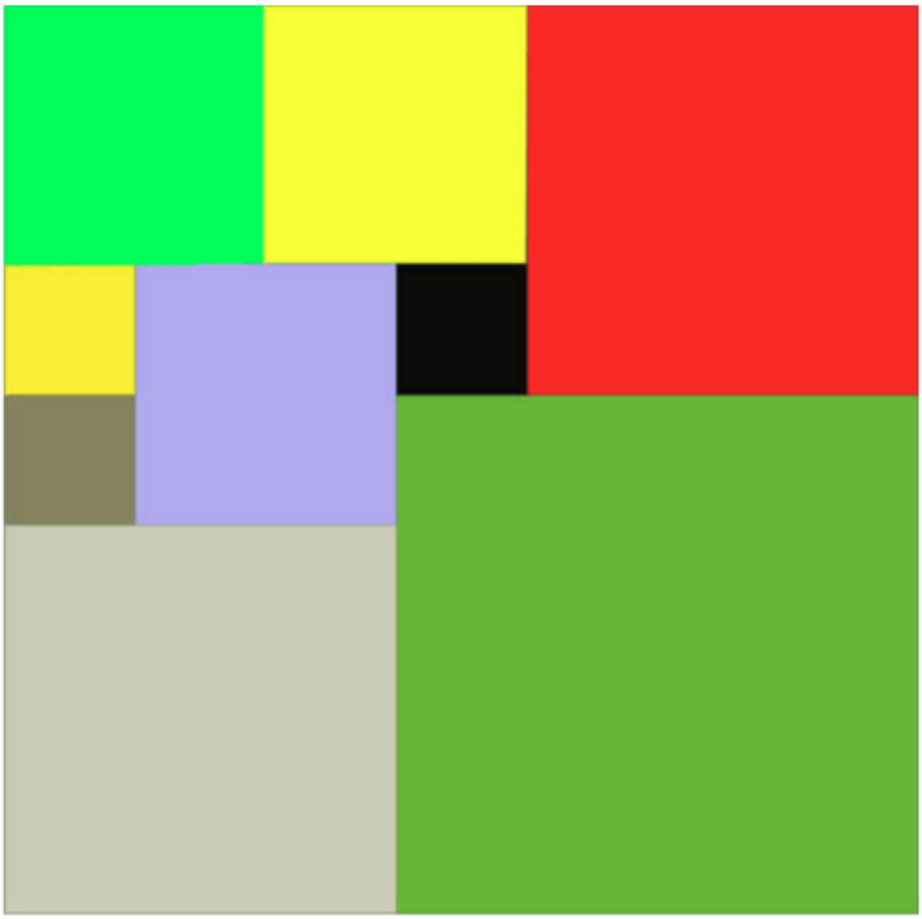
Цель работы.

Изучить и реализовать рекурсивный алгоритм поиска с возвратом и оптимизировать его для работы с большими входными данными.

Задание

У Вовы много квадратных обрезков доски. Их стороны (размер) изменяются от 1 до $N-1$, и у него есть неограниченное число обрезков любого размера. Но ему очень хочется получить большую столешницу - квадрат размера N. Он может получить ее, собрав из уже имеющихся обрезков(квадратов).

Например, столешница размера 7×7 может быть построена из 9 обрезков.



Внутри столешницы не должно быть пустот, обрезки не должны выходить за пределы столешницы и не должны перекрываться. Кроме того, Вова хочет использовать минимально возможное число обрезков.

Входные данные

Размер столешницы – одно целое число N ($2 \le N \le 20$).

Выходные данные

Одно число K, задающее минимальное количество обрезков(квадратов), из которых можно построить столешницу(квадрат) заданного размера N. Далее должны идти K строк, каждая из которых должна содержать три целых числа x, y и w, задающие координаты левого верхнего угла ($1 \leq x, y \leq N$) и длину стороны соответствующего обрезка(квадрата).

Пример входных данных

7

Соответствующие выходные данные

9

1 1 2

1 3 2

3 1 1

4 1 1

3 2 2

5 1 3

4 4 4

1 5 3

3 4 1

Вариант 1р.

Рекурсивный бэктрекинг. Выполнение на Stepik всех трёх заданий в разделе 2.

Выполнение работы:

В ходе выполнения работы было замечено, что при заполнении столешницы квадратами, можно выделить 2 основных случая для разных N, а именно для чётных и нечётных.

В первом случае можно заметить, что для заполнения столешницы всегда можно использовать только 4 квадрата, каждый со стороной равной $\frac{N}{2}$,

что является минимальным возможным решением. И также очевидно можно вычислить и координаты каждого квадрата, что позволяет не использовать бэктрекинг для этого случая, а сразу выводить результат.

Во втором случае необходимо работать ещё с двумя подгруппами, простыми и составными числами. Чтобы оптимизировать алгоритм, можно свести решение задачи с составными числами, к задаче с простыми числами. Если представить данное N как произведение просто натурального числа simp_num на другое число num ($N = simp_num \cdot num$), то квадрат со стороной N будет являться квадратом со стороной simp_num, которой увеличен в num раз. А значит все его составные части также увеличили длины своих сторон в num раз, а их количество осталось прежним. Для вычисления координат будем пользоваться формулой: $(x - 1) \cdot num + 1$.

При решении задачи с простым N, для оптимизации алгоритма, будем сразу размещать на столешнице 3 квадрата:

• 1 квадрат со стороной $\frac{N+1}{2}$ расположенный в верхнем левом углу.

• 2 квадрата со стороной $\frac{N-1}{2}$ расположенные в правом верхнем и в левом нижнем углах.

После этого для заполнения останется фигура, представляющая собой квадрат $\frac{N+1}{2}$ с вырезанной частью. Для этого будем использовать алгоритм поиска с возвратом – void backtrack(int count). В первую очередь на каждом шаге работы алгоритма проверяется то, превысило ли текущее количество размещённых квадратов count (по умолчанию 3) минимальное количество best_count (по умолчанию равняется N + 4, но может уменьшаться, если была найдена комбинация, где для заполнения столешницы требуется меньше квадратов), если да, то ничего не возвращает, иначе функция продолжает работу. Далее происходит поиск координат пустого места для размещения нового квадрата с помощью функции find_new_place (описывается далее), если его нет, то обновляется значение минимального количества квадратов best_count для размещения и контейнер best_comb, который хранит

комбинацию для этого случая. Если свободное место есть, то определяется максимальный размер квадрата max_square_size, который можно разместить так, чтобы его верхний левый угол был на полученных координатах. Далее, циклически будут проверяться все размеры квадратов в текущей позиции и комбинации для них (так как нужно найти минимальное количество, то будем брать размеры квадратов по убыванию). В цикле, с помощью функции its_empty (описывается далее) проверяется, что квадрат с выбранным размером точно можно разместить, если это так, то с помощью метода place (описывается далее) в двумерном массиве table меняются значения для координат, которые теперь заняты на 1, в контейнер current_comb записываются основные характеристики квадрата, вызывается функция бэктрекинга (с увеличением количества квадратов на 1) для поиска всех комбинаций заполнения. После этого из контейнера current_comb удаляется последний поставленный квадрат, а место, которое он занимал на столе снова инициализируется нулями.

Помимо этого, в программе была реализована 1 структура и ряд функций:

• struct Square – структура, которая используется для удобства хранения квадратов, которыми заполнена столешница, у неё есть всего 3 целочисленных поля, а именно x, y (координаты квадрата) и square_size (размер квадрата).

•int get_min(int a, int b) – нужен для выбора минимального значения.

• bool find_new_place(int &x, int &y) – используется для поиска нового места для размещения квадрата, циклически проходит сверху вниз, слева направо, пока не будет найдена не занятая клетка, если такая есть, то x и y присваиваются её координаты и возвращается True, если нет, то возвращается False.

• bool its_empty(int x, int y, int square_size) – используется для проверки на то, можно ли разместить квадрат выбранного размера в области, в нём проверяется, чтобы квадрат не выходил за границы столешницы и все клетки, которые он занимает были свободны.

• void place(int x, int y, int square_size, int val = 1) – используется для размещения квадрата на столешницы, в нём циклически заменяются все значения на val в пределах квадрата. 0 – клетка пуста, 1 – клетка заполнена. Ничего не возвращает.

• int main() – основной цикл работы программы, в нём определяется с каким из двух случаев будет вестись работа. Для чётных чисел сразу выводится результат, для составных нечётных вычисляется простое, для простых инициализируется двумерный массив, представляющий собой стол, а также размещаются 3 основных квадрата. После вызова бэктрекинга выводится результат, и программа завершает свою работу.

Для хранения частичных решений используются 3 глобальные переменные:

• int best_count – хранит минимальное количество квадратов для заполнения столешницы.

• std::vector<Square> current_comb – контейнер, который хранит структуры Square, представляющие собой квадраты для 1 комбинации.

• std::vector<Square> best_comb – контейнер, который хранит комбинацию квадратов для минимального количества.

Оценка сложности работы алгоритма для вычисления простых чисел по времени показала, что она равна $O(C^N)$ а именно является экспоненциальной.

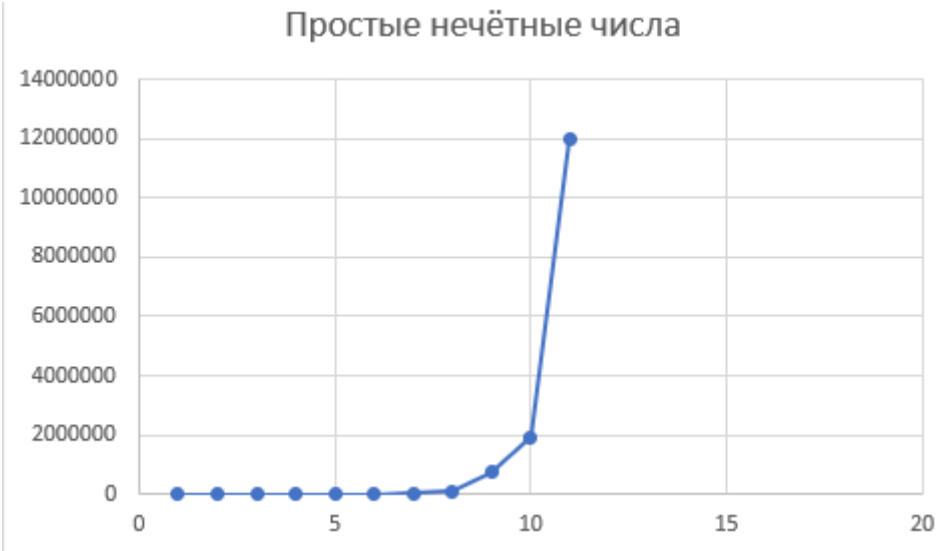


Таблица 1 – оценка сложности работы алгоритма по времени.

Оценка сложности работы алгоритма по памяти показывает, что для хранения таблицы, комбинаций квадратов и при рекурсии требуется максимум $O(n^2)$.

Исходный код программы смотри в Приложении 1.

№	Входные данные	Выходные данные	Комментарий
1	7	9 1 1 4 1 5 3 5 1 3 4 5 2 4 7 1 5 4 1 5 7 1 6 4 2 6 6 2	Работа алгоритма для задания из примера.
2	10	4 1 1 5 1 6 5 6 1 5 6 6 5	Работа алгоритма для чётного числа N.
3	11	11 1 1 6 1 7 5 7 1 5 6 7 3 6 10 2 7 6 1 8 6 1 8 10 1 8 11 1 9 6 3 9 9 3	Работа алгоритма для простого числа N.
4	35	8 1 1 21 1 22 14 22 1 14 15 22 14 22 15 7 29 15 7 29 22 7 29 29 7	Работа алгоритма для составного нечётного числа N.

5	39	6 1 1 26 1 27 13 27 1 13 14 27 13 27 14 13 27 27 13	Работа алгоритма для большого простого числа N.

Таблица 2 – тестирование работы программы.

Вывод.

Был изучен и реализован алгоритм поиска с возвратом бэктрекингом для решения задачи о подборе минимального количества обрезков для формирования столешницы. Также, для работы с большими входными был введён ряд оптимизаций, которые позволяют уменьшить время работы алгоритма и использование памяти.

ПРИЛОЖЕНИЕ 1

ИСХОДНЫЙ КОД ПРОГРАММЫ

```
#include <iostream>
#include <vector>

struct Square{
    int x, y, w;
};

int new_n;
int best_count;
std::vector<std::vector<int>> table;
std::vector<Square> current_comb;
std::vector<Square> best_comb;

int get_min(int a, int b){
    if (a < b){
        return a;
    } else {
        return b;
    }
}

bool find_new_place(int &x, int &y){
    for (int i = 0; i < new_n; i++){
        for (int j = 0; j < new_n; j++){
            if (table[i][j] == 0){
                x = i;
                y = j;
                return true;
            }
        }
    }
    return false;
}

bool its_empty(int x, int y, int square_size){
    if (x + square_size > new_n || y + square_size > new_n){
        return false;
    }
    for (int i = x; i < x + square_size; i++){
        for (int j = y; j < y + square_size; j++){
            if (table[i][j] != 0) return false;
        }
    }
    return true;
}

void place(int x, int y, int square_size, int val = 1){
    for (int i = x; i < x + square_size; i++){
        for (int j = y; j < y + square_size; j++){
            table[i][j] = val;
        }
    }
}
```

```
void backtrack(int count){
    if (count >= best_count){
        return;
    }

    int x, y;
    if (!find_new_place(x, y)){
        best_count = count;
        best_comb = current_comb;
        return;
    }

    int max_square_size = get_min(new_n - x, new_n - y);
    for (int square_size = max_square_size; square_size > 0;
square_size--){
        if (its_empty(x, y, square_size)){
            place(x, y, square_size);
            current_comb.push_back({x + 1, y + 1, square_size});
            backtrack(count + 1);

            current_comb.pop_back();
            place(x, y, square_size, 0);
        }
    }
}

int main(){
    int N;
    std::cin >> N;

    if (N % 2 == 0) {
        int h = N / 2;
        std::cout << 4 << std::endl;
        std::cout << "1 1 " << h << std::endl;
        std::cout << "1 " << h + 1 << " " << h << std::endl;
        std::cout << h + 1 << " 1 " << h << std::endl;
        std::cout << h + 1 << " " << h + 1 << " " << h << std::endl;
        return 0;
    }

    new_n = N;
    int parametr = 1;
    for (int i = 2; i <= 10; i++){
        if (N % i == 0) {
            new_n = i;
            parametr = N / new_n;
            break;
        }
    }

    table.assign(new_n, std::vector<int>(new_n, 0));
    best_count = new_n + 4;

    int square_size = (new_n + 1) / 2;
    place(0, 0, square_size);
    current_comb.push_back({1, 1, square_size});
    place(0, square_size, square_size - 1);
```

```
    current_comb.push_back({1, square_size + 1, square_size - 1});
    place(square_size, 0, square_size - 1);
    current_comb.push_back({square_size + 1, 1, square_size - 1});

    backtrack(3);

    std::cout << best_count << std::endl;
    for (int i = 0; i < best_comb.size(); ++i){
        std::cout << (best_comb[i].x - 1) * parametr + 1 << " " <<
(best_comb[i].y - 1) * parametr + 1 << " " << best_comb[i].w *
parametr << std::endl;
    }
    return 0;
}
```